SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence COURSE CODE:CSA1701

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 5 **Duration: 1 Hour**
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I Geetika T(192511469) (Name / Reg No) certify that this submission is my original work
and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Geetika T

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Question 1: AI Drones for Emergency Medicine Delivery

1.1 Understanding the scenario

A government wants drones to carry emergency medicine into a flood-affected region. Nothing about this environment stays still. A road that was open ten minutes ago can be under water now, and a storm cell can make one stretch of the route far more expensive in battery and time than another. The drone only has a rough idea of where the goal is (the straight-line distance), it receives partial and sometimes late updates about blocked paths, and the cost of moving between two points depends on terrain and weather. On top of that, delay costs lives, so the route must be found fast, and a crash costs the medicine and the drone, so it must also be reasonably safe.

In search terms, I model the region as a weighted graph. Each node is a waypoint (a grid cell in my demo), each edge is a possible flight leg, the edge weight g is the real cost of flying that leg (distance, wind, rain, battery), and the heuristic h(n) is the estimated cost still left from node n to the hospital or camp. The goal test is simply reaching the delivery point.

Element	Meaning in this scenario
State	Current drone position (waypoint) together with the currently known blocked paths
Actions	Fly to any adjacent waypoint that is not known to be blocked
Step cost g	Variable: 1 for normal flight, 4 for storm or rough terrain in the demo grid
Heuristic h(n)	Straight-line distance to the goal (Manhattan distance in the 4-move demo grid)
Goal test	Drone is at the delivery location
Uncertainty	Blocked paths appear while flying; weather changes costs; updates are partial

Key issues identified

- The heuristic ignores weather and flooded roads, so it can be misleading.
- Edge costs are not uniform, so the shortest path in distance is not the cheapest path in time or risk.
- The environment changes while the drone is moving, so any plan can become invalid mid-flight.
- Time to decide is limited, so the algorithm cannot spend long thinking before it takes off.
- A bad route has real consequences: delayed medicine, battery exhaustion or loss of the drone.

1.2 Task (i): Behaviour of Greedy Best-First Search

Greedy Best-First Search keeps a priority queue ordered only by the heuristic, f(n) = h(n). In plain words, from wherever it is, the drone always expands the waypoint that looks closest to the hospital as the crow flies. It never asks how much it has already spent to get there.

In the flood scenario this means the drone will push almost straight toward the goal. When the straight line is clear, it is very quick because it expands very few nodes. When the straight line crosses a storm zone, the drone does not notice that the zone is expensive, because the cost of the leg is not part of its score. It will happily fly through a stretch that costs four times as much, simply because the next waypoint is slightly closer in a straight line. If it hits a flooded section it backs out and tries the next best-looking node, which can lead to wandering in front of a wall.

Strengths under uncertainty	Weaknesses under uncertainty
Very fast decisions; expands few nodes, so it suits the urgency of the mission	Not optimal: it ignores the cost already paid, so it can choose an expensive route
Low memory and computation, which suits a small onboard computer	Not complete in general graphs with loops unless visited nodes are tracked
Reacts quickly when new information changes which node looks closest	Blind to weather cost and risk, so it walks into storms and dead ends
Gives a usable first answer almost immediately	A misleading heuristic (a river between drone and goal) sends it into traps

1.3 Task (ii): How A* handles the same situation

A* ranks nodes by f(n) = g(n) + h(n). The part g(n) is the actual cost already spent from the start to n, including the storm zones the drone has had to cross. The part h(n) is the estimated cost still to go. Adding them gives the estimated total cost of the best route that passes through n, so A* does not forget the price of what it has already done.

This is exactly how A* balances cost and heuristic. If a node looks close to the goal (small h) but was expensive to reach (large g), its f value is large and A* will look elsewhere. If a node is far from the goal but reachable cheaply, it still gets a fair chance. When h is admissible (never overestimates) and consistent, A* is both complete and optimal, so the first time it removes the goal from the queue, the route is the cheapest one given the information it has.

For the drone, I would make the edge cost g a combination of travel time, battery use and a risk penalty, for example g = time + w_r × risk. That way A* automatically avoids storm cells and shaky terrain even if they are shorter in distance. The price paid is that A* expands more nodes and uses more memory than greedy search, which matters when time is short.

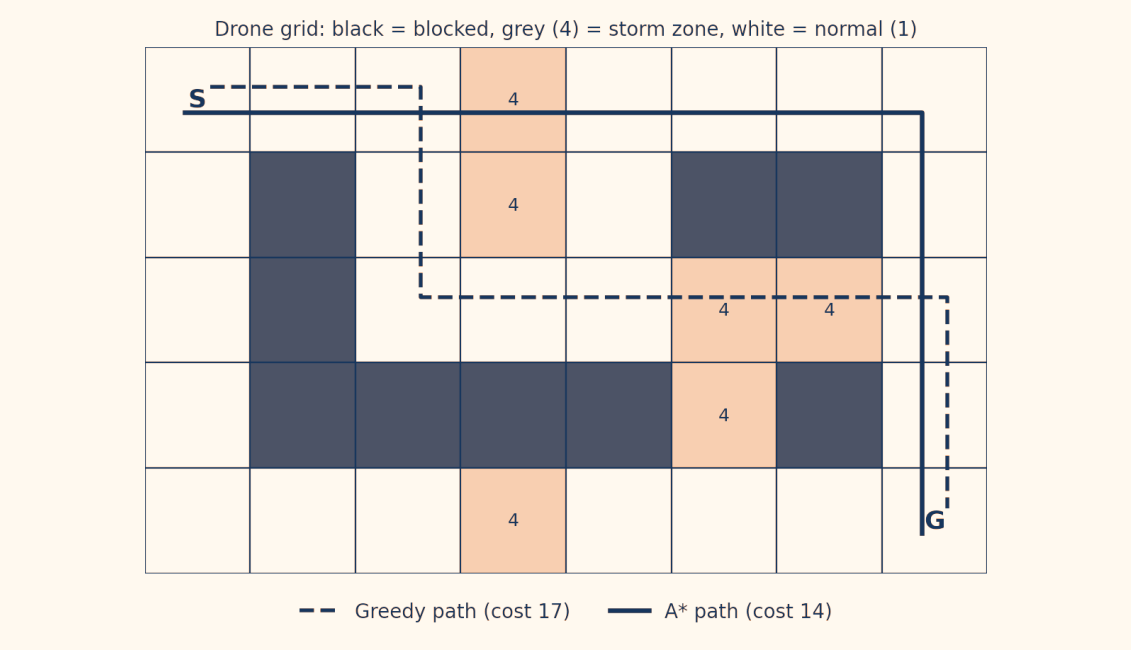


Figure 1.1: Greedy (dashed) and A (solid) routes on the same flood grid. Greedy cuts through the storm cells; A* goes around them.*

Greedy Best-First Search and A* with variable terrain cost (Python)

```python
import heapq
# '.' = normal flight (cost 1), 'w' = storm / rough terrain (cost 4),
# '#' = blocked path (flooded road, collapsed bridge)
GRID = [
    "S..w....",
    ".#.w.##.",
    ".#...ww.",
    ".####w#.",
    "...w...G",
]
COST = {'.': 1, 'S': 1, 'G': 1, 'w': 4}
def find(ch):
    for r, row in enumerate(GRID):
        if ch in row:
            return (r, row.index(ch))
def neighbours(p, blocked):
    r, c = p
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < len(GRID) and 0 <= nc < len(GRID[0]):
            ch = GRID[nr][nc]
            if ch != '#' and (nr, nc) not in blocked:
                yield (nr, nc), COST[ch]
def h(p, goal):                          # heuristic: Manhattan distance (4-move grid)
    return abs(p[0] - goal[0]) + abs(p[1] - goal[1])
def search(start, goal, blocked=frozenset(), use_g=True):
    """use_g=False -> Greedy Best-First, use_g=True -> A*"""
    pq = [(h(start, goal), 0, start, [start])]
    best_g, expanded = {start: 0}, 0
    while pq:
        _, g, node, path = heapq.heappop(pq)
        if g > best_g.get(node, 1e9):
            continue
        expanded += 1
        if node == goal:
            return path, g, expanded
        for nxt, step in neighbours(node, blocked):
            ng = g + step
            if nxt not in best_g or (use_g and ng < best_g[nxt]) or \
               (not use_g and nxt not in best_g):
                best_g[nxt] = ng
                f = ng + h(nxt, goal) if use_g else h(nxt, goal)
                heapq.heappush(pq, (f, ng, nxt, path + [nxt]))
    return None, float('inf'), expanded
if __name__ == "__main__":
    S, G = find('S'), find('G')
    for name, flag in (("Greedy", False), ("A*", True)):
        path, cost, n = search(S, G, use_g=flag)
        print(f"{name:7s} cost={cost:3d} nodes_expanded={n:2d} steps={len(path)-1}")
        print("   ", path)
    # dynamic event: a cell on the A* path becomes blocked while the drone is flying
    path, cost, _ = search(S, G)
    blocked = {path[len(path)//2]}
    mid = path[len(path)//2 - 1]
    new_path, new_cost, n = search(mid, G, frozenset(blocked))
    print("Blocked", blocked, "-> replanned from", mid, "cost", new_cost, "expanded", n)
```

Python IDLE Shell output — verified run (Python 3.12.3), Question 1

```
>>> Greedy  cost= 17 nodes_expanded=12 steps=11
    [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (2, 4), (2, 5), (2, 6), (2, 7), (3, 7), (4, 7)]
A*      cost= 14 nodes_expanded=29 steps=11
    [(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (0, 5), (0, 6), (0, 7), (1, 7), (2, 7), (3, 7), (4, 7)]
Blocked {(0, 6)} -> replanned from (0, 5) cost 14 expanded 17
>>>
```

Measure	Greedy Best-First	A*
Evaluation function	f(n) = h(n)	f(n) = g(n) + h(n)
Route cost in the demo	17	14
Nodes expanded	12	29
Optimal?	No	Yes, with an admissible heuristic
Speed to first answer	Faster	Slower
Uses weather and terrain cost?	No	Yes, through g(n)

The numbers show the trade-off honestly. Greedy finished with fewer expansions (12 against 29), but the route it picked cost 17 while A* found one costing 14, because greedy walked through two storm cells.

1.4 Task (iii): Impact of heuristic accuracy

The quality of h decides how both algorithms behave. Straight-line distance is a good heuristic on clear days because the true cost can never be less than the straight distance, so it is admissible. In a flood it is optimistic: it ignores detours around flooded

roads and the extra cost of storms. It is still safe for A* (underestimating does not break optimality), but it makes A* less focused.

Heuristic quality	Effect on Greedy Best-First	Effect on A*
Perfect (h equals true remaining cost)	Goes straight to the goal; optimal by luck	Expands only nodes on the optimal path; ideal
Close but underestimating (admissible)	Still fast but can pick costly routes	Optimal; expands a moderate number of nodes
Very weak (h close to 0)	Behaves like random or blind search	Behaves like Uniform Cost Search; optimal but slow
Overestimating (inadmissible), e.g. fear of weather added too heavily	Behaviour barely changes; it was never optimal anyway	May skip the best route, losing optimality, but can be faster
Misleading (river between drone and goal)	Gets trapped against the river, large waste	Recovers because g grows and pushes it to other branches

So greedy search is far more sensitive to heuristic errors, because it has nothing else to correct itself with. A* is more forgiving thanks to g(n). For a mission where the heuristic is known to be imperfect, a practical option is Weighted A*, $f(n) = g(n) + w \times h(n)$ with w slightly above 1 (say 1.2 to 1.5). It trades a small, bounded loss in optimality for a big saving in time.

1.5 Task (iv): Effect of dynamic changes (blocked paths)

When a path becomes blocked after the plan has been made, the plan is stale. Greedy search can react fast because it only needs to pick the next best-looking node again, but it has no memory of cost, so the recovered route may be poor. A* must repair or recompute its search tree. In my program, after the drone had flown along the A* route and cell (0, 6) was reported blocked, the drone replanned from (0, 5) and found a new route of cost 14 after expanding 17 nodes, which is quick enough for real use on a grid of this size.

Dynamic event	Impact on Greedy	Impact on A*
A path ahead is blocked	Quick re-selection, but may dead-end again	Needs replanning; result stays optimal for the new map
Storm raises cost of a region	Not noticed at all	Noticed through g, so route shifts away
Many updates in a short time	Cheap to re-run	Replanning from scratch each time becomes costly
Partial or delayed information	May commit to an invalid direction	May plan on outdated costs, so replan on every update

For frequent changes, incremental versions such as D* Lite or Lifelong Planning A* are better than running A* from scratch. They reuse the old search effort and only repair the nodes affected by the new blockage, which gives A*-quality routes at replanning speed.

1.6 Task (v): Recommended algorithm and justification

Recommendation: A* (in its replanning form, with D* Lite style repair and an optional weighted heuristic) with a risk-aware cost function.

Criterion	Greedy Best-First	A* with replanning	Winner
Real-time constraints	Very fast	Slightly slower; improved by weighted A* and incremental repair	Greedy by a small margin
Optimality of route	Not guaranteed	Optimal for the known map	A*
Safety (storms, risky terrain)	Ignores cost and risk	Risk can be added to g(n)	A*
Handling blocked paths	Quick but unreliable	Reliable after replanning	A*
Memory on the drone	Low	Higher but manageable	Greedy

My reasoning is that in a medical emergency a slightly slower decision is acceptable, but a route that flies into a storm or a flooded dead end is not. Time is saved far more by choosing a good route than by saving a few milliseconds of computation. To still respect real-time limits I would cap the planning time, use Weighted A* with w around 1.3 for the first plan, and refine the route while the drone is already flying. Greedy search can remain as an emergency fallback if the drone has to decide in a fraction of a second and no plan exists.

Summary: Greedy is quick but short-sighted, A* is slower but careful. Because the cost of a wrong route is high and the environment is variable, A* with replanning gives the best balance of speed, optimality and safety.

Question 2: Smart City Traffic Signal Optimisation

2.1 Understanding the scenario

A smart city wants one AI system to set the signal timings at hundreds of intersections at the same time. The aim is to reduce the total waiting time of vehicles, the fuel they burn while idling and the overall congestion. The difficulty is that every junction affects its neighbours: if one signal lets too many cars through, the next junction clogs. The traffic pattern also keeps changing through the day, so a timing plan that is perfect at 9 AM can be bad at 6 PM. The search space is enormous, and any method that only keeps improving the current plan can get stuck in a plan that is good locally but poor for the whole city.

Key issues identified

- A very large combinatorial search space with strongly coupled variables.
- Three objectives that conflict (waiting time, fuel, congestion) and must be combined.
- Dynamic traffic, so the best solution moves over time.
- Risk of local optima, plateaus and ridges when improving a plan step by step.
- No single known goal state; the aim is to find a good enough solution quickly.

2.2 Formulation as an optimisation problem

Component	Definition
Decision variables	For each intersection i: green duration g_i, cycle length, and offset (start time) relative to neighbours
Search space	All combinations of timings for all N intersections
Objective function	Minimise J(x) = w1 × TotalWaiting(x) + w2 × Fuel(x) + w3 × Congestion(x)
Constraints	Minimum and maximum green time (e.g. 10 to 60 s), pedestrian crossing time, no conflicting greens at the same time
Neighbourhood	Change one junction's green time by a small amount, or shift one offset
Evaluation	A traffic simulator or live sensor data gives the value of J for a candidate plan

Why traditional search is inefficient. Suppose each junction has 20 possible green durations and 6 phase patterns, which is 120 settings per junction. For 100 junctions the number of complete plans is 120^100, which is roughly 10^208. Breadth-first and depth-first search would have to store or walk through a hopeless number of states. A* needs a good admissible heuristic and a goal state, but here there is no single goal; there are only better and worse plans. Also, the path to a solution does not matter, only the final configuration, which is exactly the situation where local search is the right tool. Finally, because traffic changes, any solution found after hours of search would already be out of date.

2.3 Task (i): How Hill Climbing works and its limitations

Hill Climbing starts with some timing plan (for example last week's plan for this time of day). It looks at all neighbouring plans, meaning plans that differ in one junction by one small step, evaluates J for each, and moves to the best one if it is better than the current plan. It repeats until no neighbour is better. It keeps only the current state, so memory is tiny, and each iteration is cheap, which is attractive for a live system.

The limitation is that it only ever moves uphill (or downhill on the cost). It cannot accept a temporarily worse plan, so once it reaches a plan where every neighbour is worse, it stops, whether or not that plan is the best one available. The result depends heavily on the starting plan.

```python
import math, random
N = 30                          # intersections (the real city has hundreds)
random.seed(7)
DEMAND = [random.uniform(20, 55) for _ in range(N)]   # ideal green time (s) per junction
def cost(plan):
    """Lower is better: waiting + fuel + congestion in one weighted score."""
    total = 0.0
    for i, g in enumerate(plan):
        total += (g - DEMAND[i]) ** 2 / 20          # waiting / fuel at this junction
        total += 6 * math.sin(g / 3.0) ** 2          # cycle-clash penalty -> creates local optima
        if i > 0:                                    # congestion spills to the neighbour
            total += 0.15 * abs(g - plan[i - 1])
    return total
def random_plan():
    return [random.randint(10, 60) for _ in range(N)]
def neighbours_of(plan):
    for i in range(N):
        for d in (-1, 1):
            if 10 <= plan[i] + d <= 60:
                p = plan[:]; p[i] += d
                yield p
# ---------------- Hill climbing (steepest ascent, i.e. descent on cost) ----------------
def hill_climb(plan):
    cur, cur_c = plan, cost(plan)
    while True:
        best, best_c = None, cur_c
        for nb in neighbours_of(cur):
            c = cost(nb)
            if c < best_c:
                best, best_c = nb, c
        if best is None:                    # no better neighbour: peak, plateau or ridge
            return cur, cur_c
        cur, cur_c = best, best_c
```

2.4 Task (ii): Local maxima, plateaus and ridges in the real world

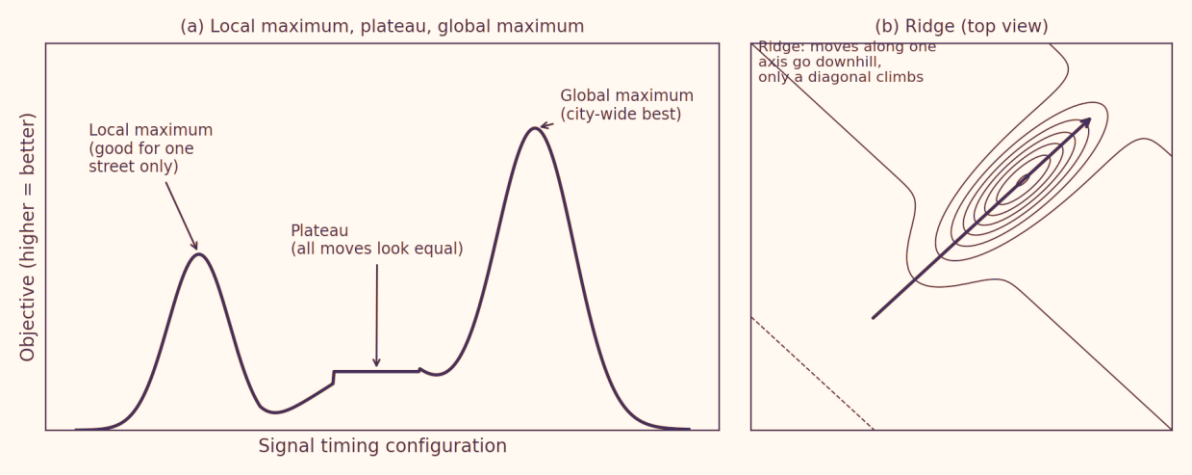


Figure 2.1: (a) Local maximum, plateau and global maximum on a one-dimensional landscape; (b) a ridge seen from above.

Problem	What happens to the algorithm	Real-world traffic interpretation
Local maximum	Every neighbour is worse, so the search stops on a small peak	The main road has a perfect green wave, but the plan starves the side roads and a better city-wide plan exists elsewhere
Plateau	Neighbours have equal value, so there is no direction to climb	Traffic is light at night; changing a green time by 1 second changes nothing, so the algorithm has no signal to follow
Ridge	Improvement needs two or more variables changed together, single changes look worse	Junction A and junction B must both shift their offset together to keep a green wave; moving only one breaks the wave and looks worse
Shoulder / noise	Flat area with a slope at the end, or noisy measurements	Random fluctuations in sensor counts can make a bad plan look temporarily good

Common patches for plain Hill Climbing are random restarts, sideways moves (to cross plateaus), and stochastic hill climbing. They help, but with hundreds of junctions random restarts are expensive, so smarter methods are needed.

2.5 Task (iii): Simulated Annealing and Genetic Algorithms

Simulated Annealing

Simulated Annealing is Hill Climbing with a controlled willingness to go the wrong way. It picks a random neighbour. If the neighbour is better it is accepted. If it is worse, it is still accepted with probability e^(-Δ/T), where Δ is how much worse it is and T is a temperature. Early on, T is high and the search wanders widely, which lets it leave local optima. As T cools, bad moves become rare and the search settles into a good region, much like metal cooling into a strong crystal. A slow cooling schedule gives a better final answer at the cost of time.

Genetic Algorithm

A Genetic Algorithm keeps a population of complete timing plans. Better plans (lower J) are more likely to be selected as parents. Crossover combines parts of two parents, for example taking the timings of the north zone from one plan and the south zone from another. Mutation randomly nudges a few timings. Over generations the population concentrates on good plans. Because it explores many regions at once, it is much less likely to be trapped, and the crossover can fix the ridge problem by moving several variables together.

```
# ---------------- Simulated annealing ----------------
def simulated_annealing(plan, T=30.0, cooling=0.995, T_min=0.01):
    cur, cur_c = plan, cost(plan)
    best, best_c = cur, cur_c
    while T > T_min:
        nxt = random.choice(list(neighbours_of(cur)))
        delta = cost(nxt) - cur_c
        if delta < 0 or random.random() < math.exp(-delta / T):   # accept worse moves sometimes
            cur, cur_c = nxt, cur_c + delta
            if cur_c < best_c:
                best, best_c = cur, cur_c
        T *= cooling
    return best, best_c
# ---------------- Genetic algorithm ----------------
def genetic(pop_size=60, generations=200, mut=0.05):
    pop = [random_plan() for _ in range(pop_size)]
    for _ in range(generations):
        pop.sort(key=cost)
        nxt = pop[:4]                                    # elitism
        while len(nxt) < pop_size:
            a, b = random.sample(pop[:30], 2)            # selection from fitter half
            cut = random.randint(1, N - 1)
            child = a[:cut] + b[cut:]                    # crossover
            child = [min(60, max(10, g + random.choice((-3, 3)))) if random.random() < mut
                     else g for g in child]              # mutation
            nxt.append(child)
        pop = nxt
    best = min(pop, key=cost)
    return best, cost(best)
if __name__ == "__main__":
    start = random_plan()
    print("start cost            :", round(cost(start), 1))
    print("hill climbing         :", round(hill_climb(start)[1], 1))
    print("simulated annealing   :", round(simulated_annealing(start)[1], 1))
    print("genetic algorithm     :", round(genetic()[1], 1))
    hc = [hill_climb(random_plan())[1] for _ in range(20)]
    sa = [simulated_annealing(random_plan())[1] for _ in range(20)]
    print("avg of 20 runs  HC:", round(sum(hc)/20,1), " SA:", round(sum(sa)/20,1),
          " best HC:", round(min(hc),1), " best SA:", round(min(sa),1))
```

Python IDLE Shell output — verified run (Python 3.12.3), Question 2

```
>>> start cost            : 592.5
    hill climbing         : 264.4
    simulated annealing   : 271.1
    genetic algorithm     : 81.7
    avg of 20 runs  HC: 262.9  SA: 238.6  best HC: 207.7  best SA: 193.9
>>>
```

Method	Final cost from one run (lower is better)	Average of 20 random starts	Best of 20 starts
Starting random plan	592.5	-	-

Hill Climbing	264.4	262.9	207.7
Simulated Annealing	271.1	238.6	193.9
Genetic Algorithm	81.7	-	-

Reading these results fairly: Hill Climbing improved the plan a lot (592.5 to 264.4) but then froze. Simulated Annealing did better on average (238.6 against 262.9) and had a better best case, though in the single run shown it ended slightly worse than Hill Climbing, which is normal because it is a random method and depends on its cooling schedule. The Genetic Algorithm reached a cost of 81.7, far lower than both, because its crossover and its larger mutation steps could jump across the barriers that trapped the one-step methods. This is a toy model, so the exact figures are not claims about a real city, but the pattern matches theory.

2.6 Task (iv): Recommended approach

Criterion	Hill Climbing	Simulated Annealing	Genetic Algorithm
Escapes local optima	No	Yes, through temperature	Yes, through population and crossover
Scalability to hundreds of junctions	Very good	Good	Good, and easy to parallelise by zone
Adaptability to changing traffic	Good for small tweaks	Moderate; needs reheating	Good; population can be reused as a warm start
Memory	Minimal	Minimal	Higher (many plans stored)
Tuning effort	Low	Cooling schedule	Population, crossover and mutation rates
Solution quality in my demo	Lowest	Better on average	Best

Recommendation: a hybrid, with a Genetic Algorithm for the global plan and Hill Climbing or Simulated Annealing for live fine tuning.

The city is divided into zones. A Genetic Algorithm, run offline or every few minutes on a simulator, produces strong base plans for each time period, since it handles the huge space and the ridge-like dependencies between neighbouring junctions. The best plans from the last run are kept as the starting population, so when traffic changes the algorithm does not start from zero. In real time, a cheap Hill Climbing step (or a low-temperature Simulated Annealing step) adjusts the plan to the latest sensor data. This combination scales because zones can be processed in parallel, and it adapts because it keeps the old good plans as seeds. Pure Hill Climbing is too easily trapped and pure Genetic Algorithm alone is too slow to react minute by minute, so neither is enough on its own.

Question 3: Autonomous Mars Rover Navigation

3.1 Understanding the scenario

An autonomous rover is dropped on Mars with a goal: reach sample sites safely. The terrain is not known in advance, so there is no complete map. The rover has a limited sensor range and can only see what is near it. Craters and rocks are real hazards, and a single wrong move can end the mission. Communication with Earth takes minutes, so Earth cannot steer it. The rover must therefore plan, move, observe and re-plan on its own, continuously, while its knowledge grows.

Key issues identified

- Unknown environment and no complete map at the start.
- Partial observability because of short sensing range, shadows and sensor noise.
- Actions are real and irreversible in some cases (falling into a crater).
- Limited battery, computing power and communication.
- Plans must be revised whenever new hazards are discovered.

3.2 Task (i): Why an online search agent is needed

An offline search agent first computes the entire plan using a complete model of the world and then executes it blindly. That only works when the model is known and complete. For the rover the model does not exist at the start, so there is nothing to plan the full route on. An online search agent interleaves computation and action: it plans a little, acts, observes the result, and then plans again. It is the natural choice when the environment is unknown, when actions have consequences that can only be learned by trying, and when time is limited.

Feature	Offline search	Online search (needed here)
Knowledge of the map	Complete before planning	Discovered while moving
When it plans	Once, before acting	Repeatedly, between actions
Handling surprises	Plan breaks down	Replans after each observation
Risk	Low if the model is right, high if it is wrong	Lower because it checks reality at every step
Suitable for Mars rover?	No	Yes

3.3 Task (ii): Partial observability and dynamic environment

Challenge	Why it is hard for the rover	How the agent copes
Limited sensing range	It cannot see a crater behind a rock until it is close	Moves in small steps and senses after every step
Sensor noise and ambiguity	A shadow may look like a crater; dust may hide a rock	Keeps a probability for each cell and treats uncertain cells as risky
Unknown goal route	No complete path can be guaranteed to exist	Plans optimistically but verifies with every observation
Dynamic changes	Sand shifts, slopes slide, lighting changes	Marks old information as possibly outdated and re-observes
Irreversible actions	A fall or stuck wheel cannot be undone	Adds safety margins and avoids any move into an unseen cell next to a hazard
Limited energy	Wandering wastes battery	Uses heuristic guidance toward the goal and avoids repeated backtracking

3.4 Task (iii): How the rover builds and updates its internal model

The rover keeps an internal map, usually an occupancy grid, where each cell has a state: unknown, free, or hazard. At the start every cell is unknown. After each move the sensors report the cells within range, and the rover writes this into the map. A more careful version stores for each cell the probability of being occupied and updates it with Bayes' rule, P(occupied | reading) proportional to P(reading | occupied) × P(occupied). Cells observed several times with consistent readings become more certain.

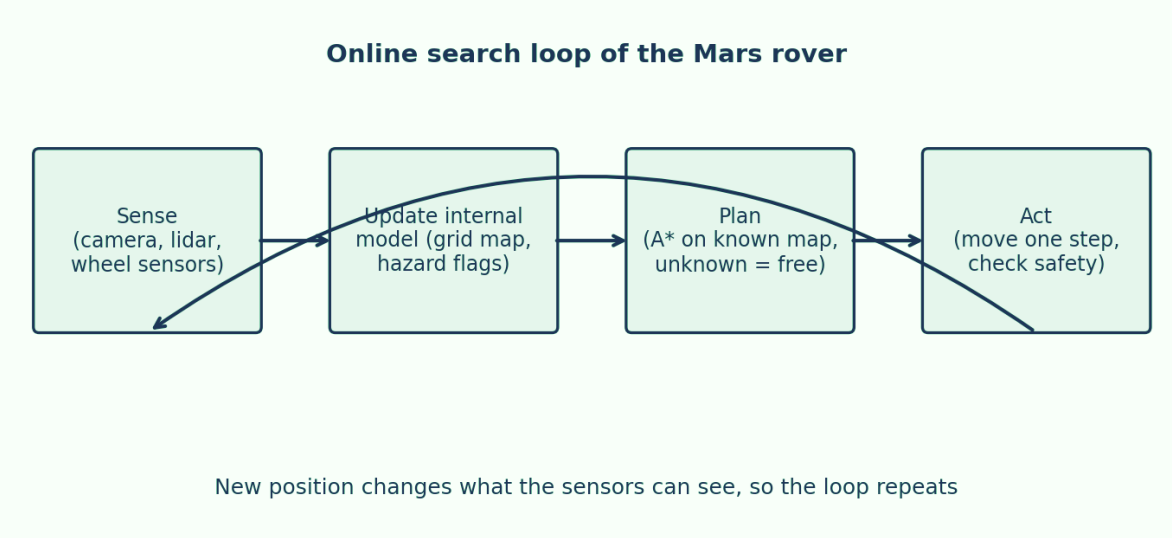


Figure 3.1: The sense, update, plan and act loop that the rover repeats at every step.

The planning step uses the current model. A common and effective rule is the free-space assumption: treat unknown cells as passable, plan the shortest route with A*, and take one step. If a new observation shows a hazard on the planned route, the plan is thrown away and recomputed from the current position. This is the idea behind D* and D* Lite, which are used in real robot navigation.

```python
import heapq
TRUE_MAP = [            # hidden from the rover: '#' = crater/rock, '.' = safe
    "...#......",
    ".#.#.####.",
    ".#.#.#....",
    ".#...#.##.",
    ".####..#..",
    "......##..",
    "#####.....",
]
ROWS, COLS = len(TRUE_MAP), len(TRUE_MAP[0])
START, GOAL = (0, 0), (6, 9)
def sense(pos, known):
    """Limited sensor: the rover only sees the 4 cells next to it."""
    r, c = pos
    known[pos] = '.'
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < ROWS and 0 <= nc < COLS:
            known[(nr, nc)] = TRUE_MAP[nr][nc]
def plan(pos, known):
    """A* on the rover's CURRENT model. Unknown cells are assumed free (optimistic)."""
    h = lambda p: abs(p[0] - GOAL[0]) + abs(p[1] - GOAL[1])
    pq, seen = [(h(pos), 0, pos, [pos])], {}
    while pq:
        f, g, node, path = heapq.heappop(pq)
        if node == GOAL:
            return path
        if node in seen and seen[node] <= g:
            continue
        seen[node] = g
        r, c = node
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < ROWS and 0 <= nc < COLS and known.get((nr, nc), '.') != '#':
                heapq.heappush(pq, (g + 1 + h((nr, nc)), g + 1, (nr, nc), path + [(nr, nc)]))
    return None
def run():
    known, pos, trail, replans = {}, START, [START], 0
    sense(pos, known)
    path = plan(pos, known)
    while pos != GOAL:
        nxt = path[1]
        if known.get(nxt) == '#':          # model changed -> plan is no longer valid
            path = plan(pos, known); replans += 1
            if path is None:
                return trail, replans, False
            continue
        pos = nxt; trail.append(pos)
        sense(pos, known)                  # update the internal model after every move
        path = path[1:]
        if any(known.get(p) == '#' for p in path):
            path = plan(pos, known); replans += 1
    return trail, replans, True
if __name__ == "__main__":
    trail, replans, ok = run()
    print("reached goal:", ok, "| moves:", len(trail) - 1, "| replans:", replans)
```

Python IDLE Shell output — verified run (Python 3.12.3), Question 3

```
>>> reached goal: True | moves: 21 | replans: 6
>>>
```

On my 7 by 10 test map the rover reached the goal in 21 moves after replanning 6 times. If a planner had known the whole map in advance, the shortest path would have been 15 moves (I checked this separately with breadth-first search). The extra 6 moves are the price of not knowing the map, and it is a normal and acceptable price, because the rover never walked into a hazard.

3.5 Task (iv): Online search compared with uninformed and informed search

Strategy	Information used	Works without a map?	Strength	Weakness for the rover
BFS (uninformed)	None	No, needs full state space	Complete, shortest steps	Huge memory; explores in all directions
DFS (uninformed)	None	No	Low memory	May wander deep into hazards, not optimal
Uniform Cost Search	Path cost g	No	Optimal for known costs	Slow, no sense of direction

Greedy Best-First (informed)	Heuristic h	No (offline)	Fast	Not optimal; can walk into traps
A* (informed)	g + h	No (offline), but its replanning form works	Optimal and efficient	Needs a model; must be re-run
Online search (LRTA*, D* Lite, online DFS)	Local observations plus learned map	Yes	Acts while learning; adapts	Not guaranteed optimal; can take extra steps

The main difference is that uninformed and informed offline algorithms assume the full problem is given, while online agents are built for discovering the problem as they go. Informed ideas are still very useful inside the online agent: the heuristic guides it toward the goal, and A* is the planner used on the partial map. So the best online agents are really a combination of the two.

3.6 Task (v): Most suitable approach and justification

Recommendation: an online agent that combines the free-space assumption with A* style replanning (D* Lite), together with safety margins and a belief map.

Requirement	How the recommended approach meets it
Uncertainty	Probabilistic occupancy map; uncertain cells treated as risky; every observation updates the model
Adaptability	Replans from the current position whenever new hazards appear; incremental repair keeps it cheap
Safety	Inflates hazards by a safety radius, refuses moves into unseen cells next to a hazard, keeps a known safe path back
Efficiency	Heuristic guides the rover toward the goal, avoiding the blind wandering of uninformed search
Limited computing	Incremental search reuses earlier work and only repairs the affected parts of the plan

I prefer this over a purely greedy or purely uninformed strategy because the cost of one wrong step on Mars is far higher than the cost of a few extra steps. The experiment above supports this: 21 moves against an offline ideal of 15, but zero hazard contacts. In a real mission I would add a rule that if the plan fails repeatedly or battery runs low, the rover retreats to the last known safe cell and waits for operator input.

Question 4: University Exam Timetable as a CSP

4.1 Understanding the scenario

A university must schedule many exams into a limited set of time slots and exam halls. Every student should be able to write all of their exams without overlap, rooms cannot be overbooked, some subjects must come before others, faculty are not available at all times, and students who need special accommodation must be placed in suitable rooms. When one change is made, several other things may break, which is why this is a classic Constraint Satisfaction Problem rather than a path search problem. We do not care how we reach the timetable, only that the final timetable breaks no rule.

Key issues identified

- Many interacting constraints of different kinds (hard and preferred).
- Growth in size: more courses and students make the search space explode.
- Some constraints apply to one exam (faculty), some to pairs (clashes), and some to groups (hall capacity).
- Late changes (a new student request or a room closure) must be absorbed without redoing everything.

4.2 Task (i): Variables, domains and constraints

A CSP is defined by the triple (X, D, C): variables, the domain of values each variable can take, and constraints on combinations of values.

Part	Definition for the timetable	Explanation
Variables X	One variable per exam: X = {AI, DBMS, OS, CN, MATHS, ML, ...}	Each exam must receive exactly one slot and hall
Domains D	Each domain is the set of pairs (time slot, hall), e.g. {(0,H1), (0,H2), (1,H1), ...}	Unary constraints shrink it: OS is not allowed in slot 0, CN is limited to the accessible hall H1
C1: No student clash	If exams A and B share a student, then slot(A) is not equal to slot(B)	Binary constraint; built from enrolment data
C2: Hall capacity	No two exams may use the same (slot, hall); the exam's student count must not exceed hall size	Global (all-different) constraint on pairs
C3: Precedence	slot(MATHS) < slot(AI) and slot(MATHS) < slot(ML)	Ordering constraint for prerequisite subjects
C4: Faculty availability	slot(exam) must be in the invigilator's available slots	Unary constraint that trims the domain in advance
C5: Special accommodation	Exams with accommodated students must use specially equipped halls (here H1)	Unary constraint on the hall part of the value

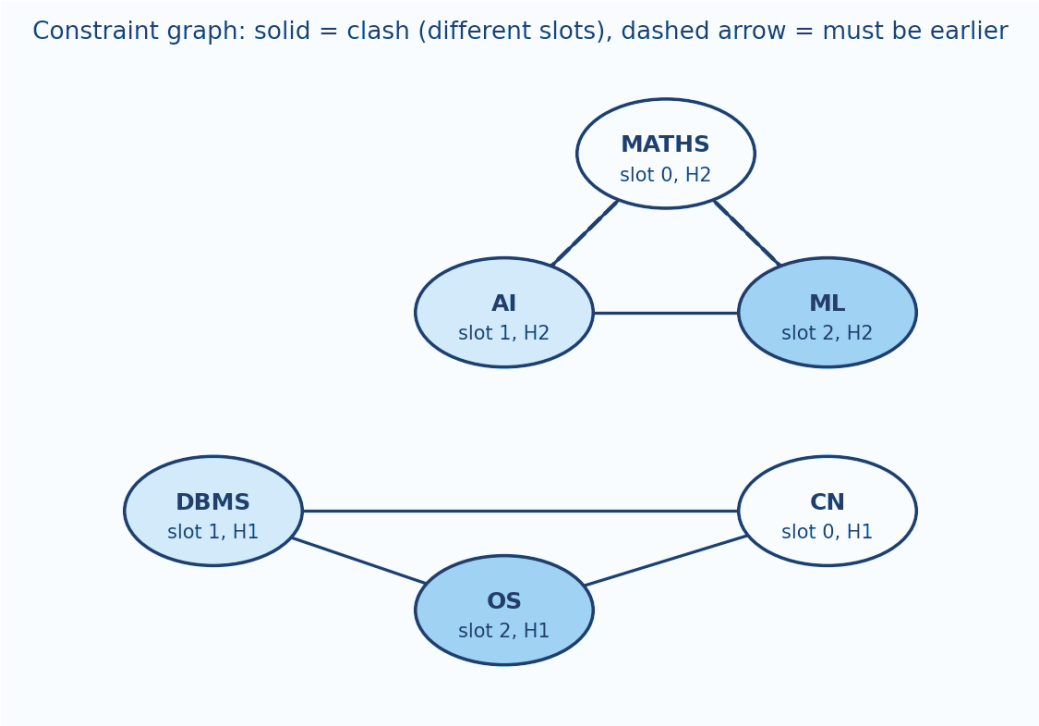


Figure 4.1: Constraint graph for the six-course demo. Each node shows the slot and hall found by the solver.

The constraint graph in Figure 4.1 has courses as nodes and clashes as edges. Dashed arrows are ordering constraints. The structure of this graph is useful: courses with many edges (like AI, ML and MATHS) are the most constrained and should be scheduled first.

4.3 Task (ii): How backtracking search works here

Backtracking is a depth-first search that builds the timetable one exam at a time.

1. Pick an unassigned exam (for example AI).
2. Try a value from its domain, such as (slot 0, H1).
3. Check it against all exams already scheduled. If any constraint is broken, reject the value and try the next one.
4. If it is consistent, move to the next exam and repeat.
5. If an exam has no value left that works, backtrack: undo the previous assignment and try another value there.
6. Stop when all exams are scheduled (success) or when all options are exhausted (no timetable exists).

Plain backtracking is correct but blind: it only discovers a conflict when it reaches the exam where it breaks, which can be many steps after the real mistake. It may therefore redo the same failing work again and again (thrashing).

4.4 Task (iii): Constraint propagation and forward checking

Forward checking improves backtracking by looking ahead. Each time an exam is given a value, the domains of the unscheduled exams are immediately filtered. If AI is placed in slot 1, then every exam that shares students with AI loses all slot-1 options. If any exam's domain becomes empty, we know right now that this choice is a dead end and backtrack at once, instead of finding out later. Stronger propagation, such as arc consistency (AC-3), removes values that cannot be part of any solution before the search even starts.

Two ordering heuristics make it better still. Minimum Remaining Values (MRV) picks the exam with the fewest legal values left, so failures show up early. Least Constraining Value (LCV) tries first the value that rules out the fewest options for the others. The degree heuristic breaks ties by choosing the exam involved in the most constraints.

```
import itertools
COURSES = ["AI", "DBMS", "OS", "CN", "MATHS", "ML"]
SLOTS   = [0, 1, 2, 3]                      # four exam sessions
HALLS   = ["H1", "H2"]                      # H1 is wheelchair accessible
CLASH = [("AI", "ML"), ("AI", "MATHS"), ("DBMS", "OS"),
         ("OS", "CN"), ("ML", "MATHS"), ("DBMS", "CN")]      # shared students
BEFORE = [("MATHS", "AI"), ("MATHS", "ML")]                  # MATHS must be earlier
NOT_SLOT = {"OS": {0}}                                       # faculty unavailable
NEEDS_H1 = {"CN"}                                            # special accommodation
def initial_domains():
    dom = {}
    for c in COURSES:
        dom[c] = [(s, h) for s in SLOTS for h in HALLS
                  if s not in NOT_SLOT.get(c, set()) and (c not in NEEDS_H1 or h == "H1")]
    return dom
def consistent(a, va, b, vb):
    if va == vb: return False                                # same hall, same slot
    if ((a, b) in CLASH or (b, a) in CLASH) and va[0] == vb[0]: return False
    if (a, b) in BEFORE and not va[0] < vb[0]: return False
    if (b, a) in BEFORE and not vb[0] < va[0]: return False
    return True
stats = {"nodes": 0}
def backtrack(assign, dom, forward_check, mrv):
    if len(assign) == len(COURSES):
        return assign
    pool = [c for c in COURSES if c not in assign]
    var = min(pool, key=lambda c: len(dom[c])) if mrv else pool[0]   # MRV heuristic
    for val in dom[var]:
        stats["nodes"] += 1
        if all(consistent(var, val, o, v) for o, v in assign.items()):
            new_dom, ok = dom, True
            if forward_check:                        # prune neighbours' domains now
                new_dom = {c: list(d) for c, d in dom.items()}
                for o in pool:
                    if o == var: continue
                    new_dom[o] = [x for x in new_dom[o] if consistent(var, val, o, x)]
                    if not new_dom[o]: ok = False; break      # dead end detected early
            if ok:
                res = backtrack({**assign, var: val}, new_dom, forward_check, mrv)
                if res: return res
    return None
if __name__ == "__main__":
    for label, fc, mrv in (("Plain backtracking", False, False),
                           ("Forward checking", True, False),
                           ("Forward checking + MRV", True, True)):
        stats["nodes"] = 0
        sol = backtrack({}, initial_domains(), fc, mrv)
        print(f"{label:24s} nodes tried = {stats['nodes']}")
    for c in COURSES: print("  ", c, "->", sol[c])
```

Python IDLE Shell output — verified run (Python 3.12.3), Question 4

```
>>> Plain backtracking       nodes tried = 1125
    Forward checking         nodes tried = 8
    Forward checking + MRV   nodes tried = 7
       AI -> (1, 'H2')
       DBMS -> (1, 'H1')
       OS -> (2, 'H1')
       CN -> (0, 'H1')
       MATHS -> (0, 'H2')
       ML -> (2, 'H2')
```

Strategy	Nodes tried (6 courses, 4 slots, 2 halls)	Remark
Plain backtracking	1125	Finds conflicts late; much repeated work
Forward checking	8	Dead ends detected immediately
Forward checking + MRV	7	Most constrained exam first; smallest search

The improvement is dramatic even on this small example: from 1125 attempts to 8, and 7 with MRV. The resulting timetable is checked below against every rule.

Exam	Slot	Hall	Rules satisfied
MATHS	0	H2	Before AI and ML (precedence)
CN	0	H1	Accessible hall for special accommodation; no clash in slot 0
AI	1	H2	After MATHS; no overlap with ML or MATHS students
DBMS	1	H1	Clashes with OS and CN, which are in other slots
OS	2	H1	Faculty not available in slot 0, so slot 2 is fine
ML	2	H2	After MATHS; different slot from AI and MATHS

4.5 Task (iv): Real-world challenges and the best strategy for scalability

Challenge	Why it matters	How to handle it
Scale (thousands of exams, tens of thousands of students)	Search space grows exponentially	Decompose by department or day; use MRV, LCV and strong propagation; run in parallel
Over-constrained problems	Sometimes no perfect timetable exists	Separate hard constraints (must hold) from soft ones (preferred) and minimise penalty
Last-minute changes	Redoing everything is not acceptable	Keep the existing timetable and repair locally with min-conflicts search
Fairness and student welfare	Back-to-back exams are unpleasant even when legal	Add soft constraints for spacing exams and limiting several exams per day
Data quality	Wrong enrolment data creates wrong clashes	Validate enrolment and room data before solving
Explainability	Staff want to know why an exam is where it is	Log which constraints fixed each choice

Recommended strategy: a hybrid of constraint propagation with backtracking for building the first complete timetable, followed by local search (min-conflicts or Simulated Annealing) for optimisation and for late changes.

The reasoning is that exact backtracking with forward checking, MRV and LCV is excellent at producing a valid timetable and at proving that a rule set is impossible, but its worst-case time is exponential. For large universities, one can first split the problem into independent groups (for example by faculty or by day), solve each with propagation and backtracking, and then use min-conflicts to clean up the remaining violations and to absorb changes. Soft preferences (spread exams, avoid early mornings) are handled by the local search stage as an optimisation score. This gives a valid answer quickly, keeps it improvable and keeps it flexible when the data changes.

Question 5: Strategic Game AI with Minimax and Alpha-Beta Pruning

5.1 Understanding the scenario

A gaming company needs an AI opponent for a real-time strategy game. It plays against human players, so every good move for the AI is a bad move for the human, which is a two-player, zero-sum, adversarial situation. The AI has to answer within strict time limits, because a slow opponent ruins the game. Each position has many legal moves (units to move, buildings to place, attacks to launch), so the game tree is gigantic, and it cannot be searched to the end. The design problem is therefore how to choose a very good move in very little time.

Key issues identified

- Adversarial decision making, because a smart human will exploit weak moves.
- Very large branching factor and depth, leading to an exponential tree.
- Hard time limit per decision.
- The game cannot be solved fully, so positions must be estimated with an evaluation function.
- Hidden information and real-time actions in true RTS games make pure Minimax an approximation.

5.2 Task (i): How Minimax models the problem

Minimax treats the game as a tree. Each node is a game state, each edge is a legal move, and the levels alternate between the two players. The AI is MAX and wants to maximise the score. The human is MIN and is assumed to play the move that minimises the AI's score. Leaf nodes (or nodes at the depth limit) get a numeric value from the evaluation function. The values are passed up: a MAX node takes the largest value of its children, a MIN node takes the smallest. At the root, the AI picks the move leading to the child with the highest backed-up value.

The central assumption is that the opponent plays optimally. This makes the AI safe, because it guarantees at least the minimax value against any opponent, but it also means it can be overcautious against a weak human. The algorithm is complete for finite trees and optimal against an optimal opponent, with time O(b^d) and space O(b × d) when implemented with depth-first recursion.

```
import math, time
leaf_count = {"minimax": 0, "alphabeta": 0}
def minimax(node, maximizing, tag="minimax"):
    if not isinstance(node, list):            # leaf: static evaluation
        leaf_count[tag] += 1
        return node
    vals = [minimax(ch, not maximizing, tag) for ch in node]
    return max(vals) if maximizing else min(vals)
def alphabeta(node, maximizing, alpha=-math.inf, beta=math.inf, tag="alphabeta"):
    if not isinstance(node, list):
        leaf_count[tag] += 1
        return node
    if maximizing:
        best = -math.inf
        for ch in node:
            best = max(best, alphabeta(ch, False, alpha, beta, tag))
            alpha = max(alpha, best)
            if alpha >= beta:                 # MIN above avoids this
                break
        return best
    best = math.inf
    for ch in node:
        best = min(best, alphabeta(ch, True, alpha, beta, tag))
        beta = min(beta, best)
        if beta <= alpha:                     # MAX has better already
            break
    return best
```

5.3 Task (ii): Computational challenges of large game trees

If b is the branching factor and d is the search depth, the number of positions is about b^d. In a strategy game, the branching factor is large because many units can act. Even a modest depth becomes impossible. The table shows how quickly the numbers grow, and how much alpha-beta can save in the best case (about b^(d/2)).

Branching factor b	Depth d	Minimax nodes b^d	Alpha-beta best case b^(d/2)
10	4	10,000	100
30	6	about 7.3 × 10^8	27,000
35 (chess-like)	10	about 2.8 × 10^15	about 5.3 × 10^7

- Time: exponential growth means the time limit allows only a shallow search.

- Evaluation cost: each leaf needs the evaluation function, which itself costs time in a complex game.
- Horizon effect: stopping at a fixed depth can hide a disaster just beyond the horizon.
- Real-time continuous actions: the state keeps changing while the AI thinks, so a long search is outdated before it finishes.
- Repeated states: the same position can be reached by different move orders, wasting effort unless a transposition table is used.

5.4 Task (iii): How Alpha-Beta pruning improves efficiency

Alpha-Beta pruning returns exactly the same decision as Minimax but skips branches that cannot affect it. It keeps two numbers during the search. **Alpha** is the best value MAX can already guarantee along the path so far. **Beta** is the best (lowest) value MIN can already guarantee. Whenever alpha is greater than or equal to beta, the remaining children of that node are irrelevant, since the player above would never allow the game to reach it, and they are cut off.

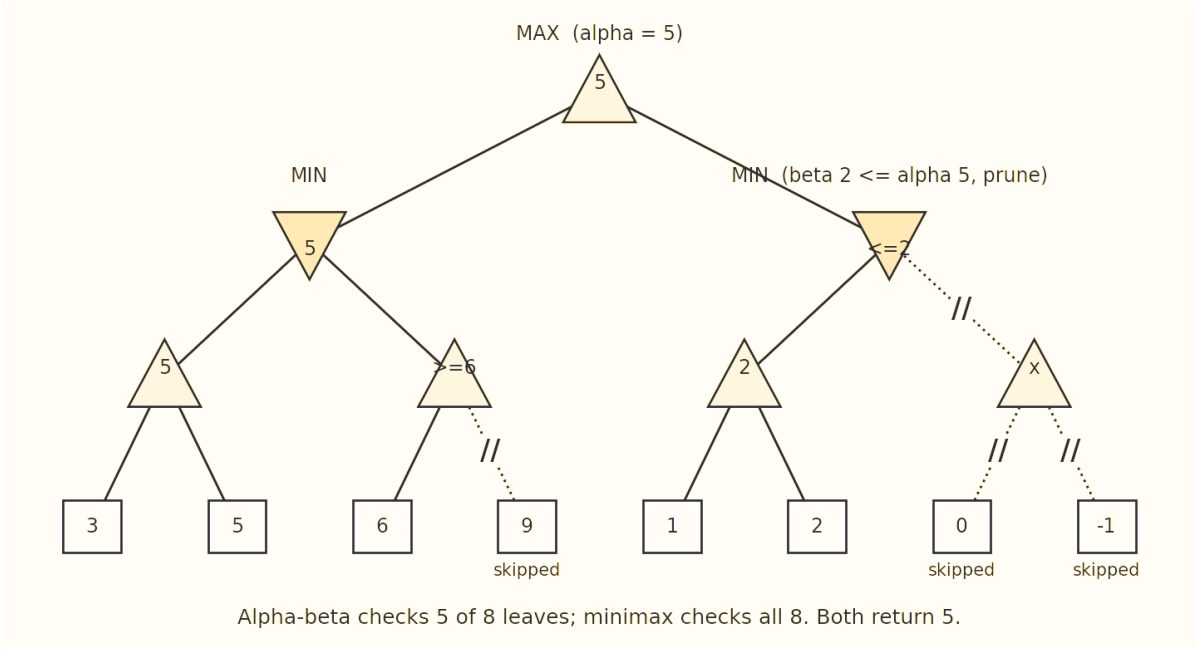


Figure 5.1: Alpha-beta on a small tree. Dotted edges marked // are pruned; only 5 of the 8 leaves are evaluated.

Step by step reasoning on the tree in Figure 5.1:

1. The search goes down the left MIN branch. The first MAX child sees leaves 3 and 5 and returns 5, so the left MIN node has a bound of at most 5 (beta = 5), and the root has alpha = 5 at least.
2. The second MAX child under it looks at leaf 6. Since 6 is already larger than beta = 5, the MIN node above would never pick this branch, so the leaf 9 is skipped. The left MIN node returns 5 and the root's alpha becomes 5.
3. In the right MIN branch, the first MAX child returns 2 (leaves 1 and 2). That makes the right MIN node worth at most 2, which is already below alpha = 5, so the root will never choose it. The whole second MAX child (leaves 0 and -1) is pruned.
4. The root value is 5, the same as full Minimax, but 3 of the 8 leaves were never evaluated.

Python IDLE Shell output — verified run (Python 3.12.3), Question 5

```
Python 3.12.3 (main, Aug 31 2026, 10:18:26) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license()" for more information.
>>> minimax value   : 5 leaves checked: 8
alpha-beta value: 5 leaves checked: 5
sample evaluation: 0.42
```

Property	Minimax	Alpha-Beta
Result	Optimal move	Same optimal move
Worst-case time	O(b^d)	O(b^d), when moves are ordered badly
Best-case time	O(b^d)	O(b^(d/2)), when best moves are tried first
Effect of move ordering	None	Very large; good ordering allows about twice the depth
Leaves checked in the example	8	5

The important practical point is move ordering. If the strongest moves are examined first (captures, attacks on key units, previous best move), cut-offs occur early and often. With good ordering alpha-beta can search roughly twice as deep as Minimax in the same time, which is a huge gain in playing strength.

5.5 Task (iv): Designing an evaluation function

Since the AI cannot reach the end of the game, it needs a static evaluation function that estimates how good a position is for it. A standard approach is a weighted linear combination: Eval(s) = w1·f1(s) + w2·f2(s) + ... + wn·fn(s), where each feature fi is normalised between -1 and +1 from the AI's point of view. A positive value favours the AI.

Feature	What it measures	Weight in my design	Why it matters
Material (army strength)	Value of own units minus enemy units, weighted by unit type and health	1.0	Winning fights is the most direct route to winning the game
Economy	Resource income, workers and stored resources	0.6	Money turns into future units, so it predicts later strength
Territory and map control	Share of map, expansions and key positions held	0.4	Control gives vision, income and safe movement
Technology	Upgrades and unlocked unit types	0.3	Better tech gives long-term advantage
Threat to base	Enemy strength near own base or key buildings	-0.8	Penalty for danger; losing the base ends the game

For example, a position with material 0.5, economy 0.2, territory 0.1, technology 0.0 and threat 0.3 gets 0.5 + 0.12 + 0.04 + 0 - 0.24 = 0.42, which the program confirms. Good evaluation functions have three qualities: they are fast to compute, they agree with the real chance of winning, and they are weighted according to what matters most. The weights should not be guesses for long: they can be tuned by self-play, by regression against game outcomes, or by evolutionary search. Terminal states override the formula (a win is +infinity, a loss is -infinity), and quiescence search, which keeps searching during sudden fights before evaluating, avoids misjudging a position in the middle of a battle.

5.6 Task (v): Balancing optimality and real-time performance

Recommended strategy: Alpha-Beta search with iterative deepening, a hard time budget, good move ordering and a transposition table, backed by an evaluation function and, where the branching factor is still too large, a Monte Carlo Tree Search component.

Program 5b: Evaluation function and time-limited iterative deepening (Python)

```
# ---------- evaluation function for the RTS game ----------
WEIGHTS = dict(material=1.0, economy=0.6, territory=0.4, tech=0.3, threat=-0.8)
def evaluate(s):
    """s = dict of normalised features (-1..+1) seen from the AI's side."""
    return sum(WEIGHTS[k] * s[k] for k in WEIGHTS)
# ---------- iterative deepening under a time limit ----------
def best_move(state, moves, apply, depth_search, time_limit=0.05):
    start, best, depth = time.time(), None, 1
    while time.time() - start < time_limit:
        scored = [(depth_search(apply(state, m), depth), m) for m in moves(state)]
        best = max(scored, key=lambda x: x[0])[1]      # keep last COMPLETED depth
        depth += 1
    return best
if __name__ == "__main__":
    tree = [[[3, 5], [6, 9]], [[1, 2], [0, -1]]]
    print("minimax value   :", minimax(tree, True),   "leaves checked:", leaf_count["minimax"])
    print("alpha-beta value:", alphabeta(tree, True), "leaves checked:", leaf_count["alphabeta"])
    sample = dict(material=0.5, economy=0.2, territory=0.1, tech=0.0, threat=0.3)
    print("sample evaluation:", round(evaluate(sample), 2))
```

Technique	What it does	Benefit for real-time play
Iterative deepening	Searches depth 1, then 2, then 3 and so on until time runs out	Always has a complete answer ready; uses the time budget fully
Move ordering	Tries previous best moves, captures and threats first	Gets close to the b^(d/2) best case of pruning
Transposition table	Stores values of positions already seen	Avoids repeating work from different move orders
Depth limit plus evaluation	Replaces the unreachable game end with an estimate	Makes the problem finite

Quiescence search	Extends search in unstable positions	Reduces the horizon effect
Selective pruning / beam	Considers only the most promising few moves	Cuts branching when b is huge
Opening books, scripted macros	Pre-computed moves for common situations	Saves time at the start and for routine actions
Difficulty scaling	Limits depth or adds randomness for beginners	Keeps the game fun for different players

The idea is to accept a small loss in theoretical optimality in exchange for answers that always arrive on time. Since alpha-beta does not change the result of Minimax, the only approximation is the depth limit and the evaluation function. Iterative deepening guarantees that when the clock runs out the AI plays the move from the deepest completed search, and the previous iteration's best move is tried first next time, which improves pruning. For very large RTS action spaces, Monte Carlo Tree Search or a learned policy can pick a short list of candidate moves, and alpha-beta then checks them tactically. This hybrid is flexible, safe against strong opponents, and scalable to the time budget.

Overall Summary and Comparison

The five scenarios cover the main families of techniques in this course outcome. The table below puts the decisions side by side so that the reasoning behind each choice is easy to see.

Question	**Problem type**	**Main techniques studied**	**Final recommendation**	**Main reason**
1. Drone delivery	Path search in a dynamic, weighted graph	Greedy Best-First, A*	A* with replanning (D* Lite, weighted A*) and risk-aware costs	Balances optimality and safety; acceptable speed
2. Traffic signals	Large optimisation problem	Hill Climbing, Simulated Annealing, Genetic Algorithm	Hybrid: GA for global plan, local search for live tuning	Escapes local optima and scales by zones
3. Mars rover	Search with unknown, partially observed world	Online search, A* replanning	Online agent with free-space assumption and safety margins	Adapts to new information without risking the rover
4. Exam timetable	Constraint satisfaction	Backtracking, forward checking, MRV, LCV	Propagation with backtracking, then local search for repairs	Valid timetables fast; flexible for late changes
5. Strategy game AI	Adversarial search	Minimax, Alpha-Beta, evaluation functions	Alpha-Beta with iterative deepening and time budget, plus MCTS if needed	Best play within strict time limits

Across all five problems the same lesson keeps appearing: the right algorithm depends on what the environment is like. When the cost of mistakes is high and the world changes, careful methods with replanning win over fast but short-sighted ones. When the search space is huge and there is no single goal state, local and population-based search is more practical than systematic search. When the problem has rules that must never be broken, constraint propagation does the heavy lifting. And when an intelligent opponent is involved, pruning and good evaluation allow deeper thinking in the same amount of time.